

Senior Ontology - Smart Route & User Management API

Unit test specification: User Registration (1.0) and Login (2.0)

Scope note: Backend tests target **server.py** (FastAPI). DFD processes 1.1–1.3 and 2.1–2.2 are mapped to API endpoints and mocked **User Info** store. Flutter UI validation (field format on device) is out of scope for these Python unit tests.

Test suite A — Process 1.0 User Registration

Test Case ID	TC-REG-1.0
Python module	UnitTest_User_Login_Validation.py
Priority	High
Data store	User Info (Key.json / mocked read_users, save_users)

DFD sub-processes covered

1.1 Get user information — Five fields (firstname, lastname, email, phonenumber, password) are accepted by **POST /signup** via model **UserRegister**. **Test:** *test_signup_success_and_change_password_branches* (signup payload).

1.2 Check existing user — Duplicate email rejected before save. **GET /check-email** exposes existence check for UI. **Tests:** *test_signup_duplicate_email_returns_400*, *test_root_health_and_check_email* (check_email branch).

1.3 Record user information — On success, **save_users** persists the new row. **Test:** *test_signup_success_and_change_password_branches* (store length 1 after signup).

Step	Action	Expected Result	Status
1	POST /signup with full UserRegister	201, User created	Pass
2	POST /signup duplicate email	400, Email already registered	Pass
3	GET /check-email existing vs missing	exists true/false	Pass

DFD / requirement	Unit test method
1.1–1.3 Registration flow	test_signup_*, test_root_health_and_check_email

Test suite B — Process 2.0 Login

Test Case ID	TC-LOGIN-2.0
Python module	UnitTest_User_Login_Validation.py
Priority	High
Data store	User Info (mocked)

DFD sub-processes covered

2.1 Get user information — Email and password submitted as **LoginRequest**.

2.2 Check existing user — Server scans store; match on email (case-insensitive) and exact password.

2.0 Login completion — Returns **User Info** subset (firstname, email) to client on success.

Step	Action	Expected Result	Status
1	POST /login valid credentials	200, Success + user	Pass
2	POST /login case-insensitive email	200, Success	Pass
3	POST /login wrong password	401, Invalid credentials	Pass
4	POST /login unknown email	401, Invalid credentials	Pass

DFD / requirement	Unit test method
2.1–2.2 Input + validation	all test_login_* methods
2.0 Success response	test_login_success_returns_payload, test_login_email_case_insensitive
2.2 Failure paths	test_login_wrong_password_returns_401, test_login_unknown_email_returns_401

Related (post-registration): Change password flows — *test_signup_success_and_change_password_branches* (401/404/success).

Coverage: run **backend_tests/run_coverage.sh** — UnitTest_User_Login_Validation.py 100%. Generated 2026-04-06 10:40.